Handwritten Digit Generation Using cDCGAN and cGAN On MNIST Dataset

1. Introduction & Motivation

Generative AI has demonstrated remarkable progress in synthesizing realistic images, audio, and text. Among these advancements, Generative Adversarial Networks (GANs) have become a powerful technique for producing high-quality synthetic images. This project focuses on applying GANs to a classic benchmark problem: the controlled generation of handwritten digits by training and comparing two architectures a Conditional GAN [1](cGAN) and a Conditional Deep Convolutional GAN [1](cDCGAN [3])—on the MNIST dataset.

The primary goal of this project is to build a system capable of generating realistic handwritten digits based on user-specified inputs. For example, when the user enters a single digit like "7" or a multi-digit sequence such as "124," the system generates corresponding handwritten-style digits. This controlled image synthesis is achieved by conditioning the GAN [1]on the digit labels, allowing the generator to produce images that precisely match the class specified by the user.

The MNIST dataset, consisting of 70,000 grayscale handwritten digit images (28 x 28 pixels), provides a simple yet effective benchmark for generative modeling. It is widely used for evaluating models that learn low-level visual patterns such as strokes, curvature, and writing style, making it an ideal choice for training cGAN [2]s.

This project also incorporates an interactive Streamlit-based web application to demonstrate real-time generation. Users can input digits, trigger the model, and visualize the generated handwritten digits instantly. Streamlit enables rapid prototyping and seamless integration with Python-based machine learning workflows, making it suitable for deploying the generative model in a lightweight, accessible environment.

2. Dataset and Preprocessing

This project uses the MNIST Handwritten Digit dataset, which contains 70,000 grayscale images of digits ranging from 0 to 9, each sized 28×28 pixels. The dataset is loaded directly using the PyTorch torchvision.datasets.MNIST API to ensure standardized formatting and reproducibility.

For the convolution-based conditional GAN [1](cDCGAN [3]), images undergo the following transformations:

- Resize: Images are resized from 28×28 to 32×32 pixels to match the dimensional requirements of transposed convolution layers.
- Tensor Conversion: Images are converted into PyTorch tensors with shape (1, 32, 32).
- Normalization: Pixel values are normalized to the range [-1, 1] using Normalize(mean=0.5, std=0.5), which is compatible with the generator's tanh output activation.

The DataLoader is configured with a batch size of 128 and shuffling enabled.

For the fully-connected conditional GAN [1](cGAN), preprocessing is slightly simpler since the architecture operates on flattened vectors:

- Original Resolution: Images remain at the original size of 28×28.
- Tensor Conversion: Tensor format (1, 28, 28) is used.
- Normalization: Pixel values are normalized to [-1, 1].

A batch size of 128 and shuffling are also used for this model.

Both models use one-hot encoded labels to enable conditioning:

- In cDCGAN [3], labels are represented both as one-hot vectors and as spatial label maps (channels filled with 0s and 1s), allowing the discriminator and generator to integrate label information in the convolutional layers.

- In cGAN [2], labels remain in vector form because the fully-connected architecture does not require spatial reinterpretation.

3. Model Architecture & Training Details

3.1 Fully-Connected cGAN [2] [2] Architecture (cGAN)

The cGAN [2] [2] uses fully-connected (dense) network layers as a baseline. The conditional aspect is implemented by concatenating the one-hot encoded label (10 dimensions) with the input to both the Generator and the Discriminator.

Generator (G)

The Generator takes a 100-dimensional random noise vector (z) and a 10-dimensional one-hot label vector (y) as input.

- Input: z (noise, 100 dimensions) and y (label, 10 dimensions).
- Layer Flow (Path 1: Noise): z is followed by $\mathrm{FC}(100 \to 256)$, then $\mathrm{BatchNorm1d}$, then ReLU.
- Layer Flow (Path 2: Label): y is followed by $\mathrm{FC}(10 \to 256)$, then $\mathrm{BatchNorm1d}$, then ReLU.
- Layer Flow (Combined): The outputs from both paths are concatenated to form a 512-dimensional vector. This is followed by:
    - $\mathrm{FC}(512 \to 512)$, $\mathrm{BatchNorm1d}$, ReLU
    - $\mathrm{FC}(512 \to 1024)$, $\mathrm{BatchNorm1d}$, ReLU
    - $\mathrm{FC}(1024 \to 784)$, Tanh
- Output: x' (flattened 28×28 image, 784 dimensions).

Discriminator (D)

The Discriminator takes a flattened 28×28 image (784 dimensions) and the 10-dimensional one-hot label vector as input.

- Input: x (real/fake image, 784 dimensions) and y (label, 10 dimensions).
- Layer Flow (Path 1: Image): x is followed by $\mathrm{FC}(784 \to 1024)$, then $\mathrm{Leaky\ ReLU}(0.2)$.
- Layer Flow (Path 2: Label): y is followed by $\mathrm{FC}(10 \to 1024)$, then $\mathrm{Leaky\ ReLU}(0.2)$.

- Layer Flow (Combined): The outputs from both paths are concatenated to form a 2048-dimensional vector. This is followed by:
    - $\text{FC}(2048 \to 512)$, BatchNorm1d, $\text{Leaky ReLU}(0.2)$
    - $\text{FC}(512 \to 256)$, BatchNorm1d, $\text{Leaky ReLU}(0.2)$
    - $\text{FC}(256 \to 1)$, Sigmoid
- Output: A scalar probability score.

3.2 Conditional Deep Convolutional GAN [1]Architecture (cDCGAN [3])

The cDCGAN [3] uses convolutional layers for the Discriminator and transposed convolutional layers for the Generator to capture spatial image features.

Generator (G)

The Generator uses transposed convolutions to upsample the latent space. The 10-dimensional one-hot label is reshaped and concatenated with the 100-dimensional noise vector after the initial deconvolution.

- Input: z (noise, reshaped to $100 \times 1 \times 1$) and y (label, reshaped to $10 \times 1 \times 1$).
- Layer Flow (Path 1: Noise): z is followed by $\text{ConvTranspose2d}(100 \to 256)$, then BatchNorm2d, then ReLU.
- Layer Flow (Path 2: Label): y is followed by $\text{ConvTranspose2d}(10 \to 256)$, then BatchNorm2d, then ReLU.
- Layer Flow (Combined): The outputs from both paths are concatenated (512 channels total). This is followed by:
    - $\text{ConvTranspose2d}(512 \to 256)$, BatchNorm2d, ReLU
    - $\text{ConvTranspose2d}(256 \to 128)$, BatchNorm2d, ReLU
    - $\text{ConvTranspose2d}(128 \to 1)$, Tanh
- Output: x' (generated image, $1 \times 32 \times 32$).

Discriminator (D)

The Discriminator uses standard convolutions. The input image is first resized to 32×32. The label conditioning uses a special "filled" label map (y_{fill}), which is a 32×32 image of the one-hot label, concatenated with the image's feature maps after the first convolutional layer.

- Input: x (real/fake image, $1 \times 32 \times 32$) and y_{fill} (filled label map, $10 \times 32 \times 32$).
- Layer Flow (Path 1: Image): x is followed by $\text{Conv2d}(1 \to 64)$, then $\text{Leaky ReLU}(0.2)$.
- Layer Flow (Path 2: Label): y_{fill} is followed by $\text{Conv2d}(10 \to 64)$, then $\text{Leaky ReLU}(0.2)$.
- Layer Flow (Combined): The outputs from both paths are concatenated (128 channels total). This is followed by:
    - $\text{Conv2d}(128 \to 256)$, BatchNorm2d, $\text{Leaky ReLU}(0.2)$
    - $\text{Conv2d}(256 \to 512)$, BatchNorm2d, $\text{Leaky ReLU}(0.2)$
    - $\text{Conv2d}(512 \to 1)$, Sigmoid
- Output: A scalar probability score.

3.3 Loss Function (BCELoss)

Both the fully-connected cGAN [2] and the cDCGAN [3] employ the Binary Cross-Entropy (BCE) loss function $(\mathrm{BCELoss})$ to establish the adversarial minimax game. The loss is computed based on the output of the Discriminator (D), which is trained as a binary classifier.

The **Discriminator's loss** $(\mathcal{L}_{\mathrm{D}})$ is minimized to correctly classify real images (x) as true (target label 1) and generated images $(\mathrm{G}(z))$ as false (target label 0):

$$\mathcal{L}_{\mathrm{D}} = -\mathbb{E}_{x \sim p_{\mathrm{data}}}[\log \mathrm{D}(x)] - \mathbb{E}_{z \sim p_z}[\log(1 - \mathrm{D}(\mathrm{G}(z)))]$$

The Generator's loss $(\mathcal{L}_{\mathrm{G}})$ is minimized to maximize D's error on generated samples. This is achieved by setting the target label for D's output on $\mathrm{G}(z)$ to 1, effectively training G to produce samples that D classifies as real:

$$\mathcal{L}_{\mathrm{G}} = -\mathbb{E}_{z \sim p_z}[\log \mathrm{D}(\mathrm{G}(z))]$$

3.4 Shared Training Details

Parameter	**Value**	**Notes**
Batch Size	128	
Optimizer	Adam	
Initial Learning Rate (lr):	0.0002	For both G and D.
Adam Betas	$(0.5, 0.999)$	
Loss Function	Binary Cross Entropy $(\mathrm{BCELoss})$	Standard for GANs.
Weight Initialization	Gaussian $\mathcal{N}(0, 0.02)$	

Data Preprocessing	Normalize to $[-1, 1]$	Using mean=0.5, std=0.5.

Specific Training Schedules

- cDCGAN [3] (20 Epochs): The learning rate was decayed by a factor of 10 at epochs 11 and 16. The total training time was approximately 24.1 minutes, averaging ≈ 70 seconds per epoch.
- cGAN (20 Epochs Evaluated): The provided code was set to train for 50 epochs, but the evaluation was based on 20 epochs. The total training time for 20 epochs was approximately 6.31 minutes, averaging ≈ 16.8 seconds per epoch.

3.5 Comparison of Training Stability

Model	**Total Epochs Evaluated**	**Total Training Time (20 Epochs)**	**Average Epoch Time**	**Stability (Qualitative)**
cDCGAN [3]	20	24.1 minutes	70 seconds	**Stable:** Smooth loss decrease (D) and expected gradual rise (G).
cGAN	20	6.31 minutes	16.8 seconds	**Unstable:** Loss fluctuations and oscillations in both G and D.

The cDCGAN [3] demonstrated superior training stability, with loss curves indicative of a successful and stable adversarial process. In contrast, the simpler fully-connected cGAN [2] exhibited unstable loss behavior, which is correlated with its qualitatively inferior, blurry, and inconsistent generated images.

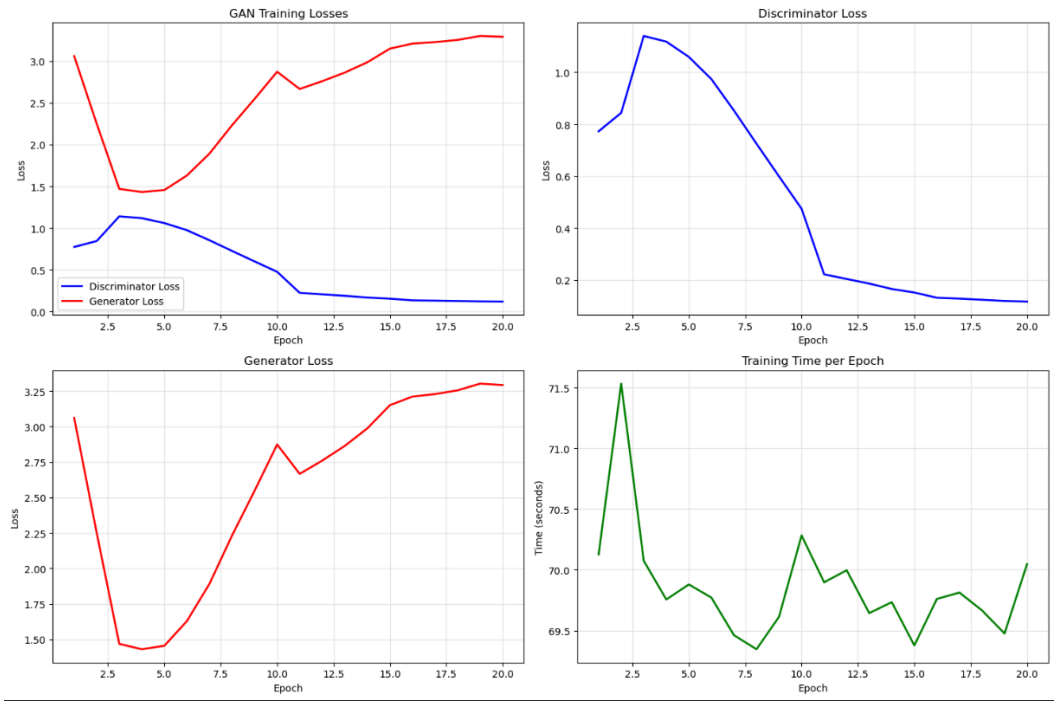
4. Evaluation

4.1 cDCGAN [3]

The model was trained for a total of 20 epochs then later at 50 epochs. The full training process required approximately 24.1 minutes, with each epoch taking about 70 seconds on average. Training times were highly consistent across epochs, varying by less than half a second, which indicates that the data-loading pipeline and GPU utilization remained stable throughout training.

4.1.1 Loss Behaviour

The discriminator loss started at 0.77 and steadily decreased to 0.12 by the final epoch. This gradual reduction shows that the discriminator consistently improved at distinguishing between real and generated images. No spikes, oscillations, or signs of training collapse were observed. In contrast, the generator loss began at 3.06, initially decreased during the early epochs, and then gradually increased toward the end, finishing at 3.29. This pattern is common in GAN [1]training. As the discriminator becomes stronger, the generator must produce increasingly detailed samples to “fool” it, which often causes the generator’s loss to rise even when image quality is improving. Therefore, the generator loss cannot be interpreted in isolation; instead, it should be viewed alongside the generated samples and discriminator behavior.



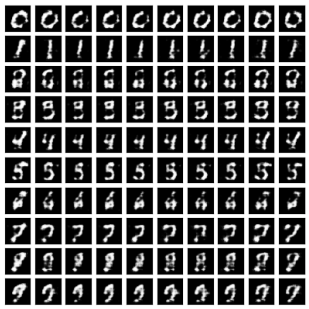
4.1.2 Qualitative Evaluation

To better understand the model’s progression, sample images were generated at the end of every epoch. These samples allow visual comparison of how the generator improves over time. During the earliest epochs, the generated digits are blurry and loosely formed. The model captures the rough shape of digits but often lacks sharpness and clear boundaries. Conditioning is partially effective at this stage, although some digits appear visually similar due to inconsistent pixel intensities.

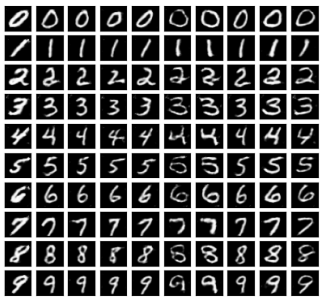
By the middle of training (around epochs 6–12), the digits become significantly clearer. The shapes of individual digits are more stable, the edges become sharper, and the overall structure begins to resemble actual MNIST samples. Conditioning is more reliable, and each generated digit corresponds more closely to its intended class.

In the final stages of training, the generator produces high-quality handwritten digits with smooth curves and well-defined structure. The conditioning mechanism works effectively: each class

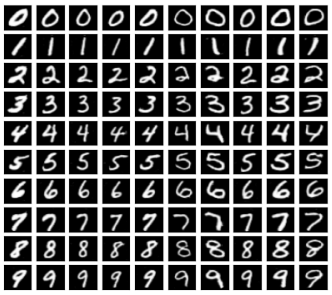
from 0 to 9 is rendered distinctly, and the model demonstrates noticeable diversity in style within each digit category. Minor artifacts remain in a few samples, but overall image quality is strong, especially for a model trained for only 20 epochs.
Images corresponding to these stages (early, mid, and final epochs) are included in the report to illustrate the model's progression visually.



Epoch 1



Epoch 4



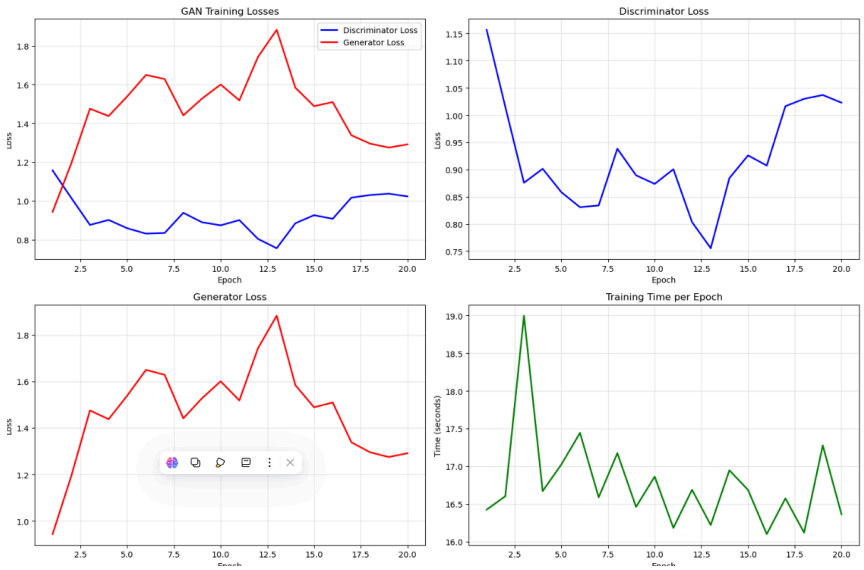
Epoch 12

4.2 cGAN [2]

The fully-connected conditional GAN [1](cGAN) was trained for 20 epochs and 50 epochs, with the entire training process finishing in approximately 6.31 minutes. Each epoch required about 16.8 seconds on average, making this model significantly faster than the convolutional cDCGAN [3] due to the lower computational cost of dense layers. Epoch times remained consistent throughout training, varying by roughly half a second, which indicates a stable training pipeline and efficient data loading.

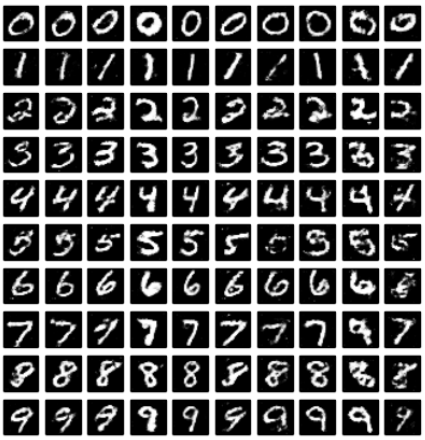
4.2.1 Loss Behaviour

The discriminator loss began at 1.15 and decreased modestly to 1.02 by the final epoch. Although a slight downward trend is present, the loss does not show the strong, steady decline observed in the cDCGAN [3]. Instead, the loss fluctuates noticeably between epochs, suggesting that the discriminator struggled to consistently distinguish real samples from generated ones.
The generator loss starts at 0.94, increases through the middle of training, peaks at 1.88 around epoch 13, and then settles to 1.29 by epoch 20. This behaviour reflects unstable adversarial dynamics: the generator initially performs well, but as training continues, it becomes less capable of reliably producing images that fool the discriminator. Unlike in the cDCGAN [3], the generator loss does not reflect a smooth learning trajectory, and the oscillations indicate inconsistent improvements in image quality from epoch to epoch.



4.2.2 Qualitative Evaluation

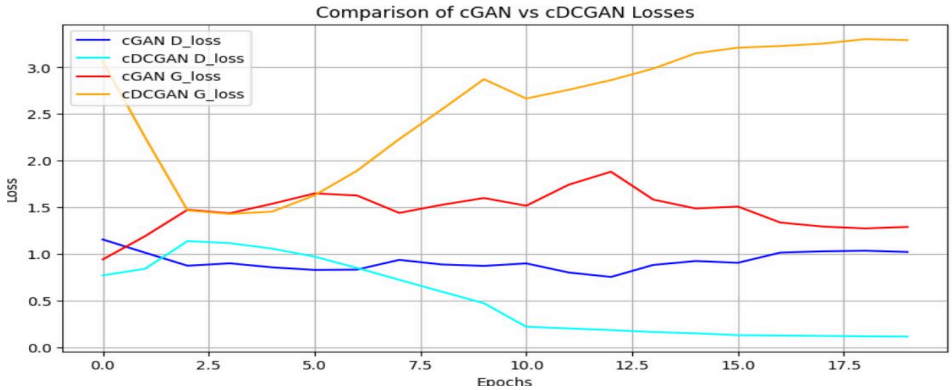
To assess output quality, generated digit images were saved at the end of each epoch. These images allow visual inspection of the model's learning progression and highlight its limitations.
In the earliest epochs, the generated digits appear fuzzy and lack coherent structure. Although some samples begin to resemble basic digit shapes, they are inconsistent and often distorted. Conditioning on specific labels is only partially effective at this stage, with several digits failing to match their target classes.
During the middle portion of training, the generator produces outputs that more closely resemble digits, but the improvements are uneven. Certain epochs produce passable results, while others regress in quality. Digits still lack well-defined edges, and many outputs include noise or irregular artifacts. The lack of spatial patterns which fully-connected layers struggle to model becomes especially apparent here.
By the end of training, the final outputs show some refinement but remain significantly inferior to the convolution-based model. The digits are often recognizable but lack sharpness, structural consistency, and stylistic diversity. Conditioning accuracy improves somewhat, but misclassified or ambiguous digits still appear. These results suggest that the model cannot capture the spatial correlations necessary for realistic handwritten digit synthesis.
Images from early, middle, and final epochs are included in the report to illustrate these performance characteristics.



Epoch 50

4.3 Summary

Compared to the convolutional cDCGAN [3], the fully-connected cGAN [2] demonstrates weaker quantitative stability and noticeably lower qualitative performance. Although it trains faster and achieves reasonable output for a simple architecture, the absence of convolutional layers prevents it from learning the spatial structure and fine-grained patterns present in handwritten digits. As a result, it produces blurrier, less consistent images and shows greater instability in both generator and discriminator losses. Overall, while the model functions correctly as a baseline conditional GAN, it is clearly outperformed by the cDCGAN [3] in both realism and conditioning accuracy.



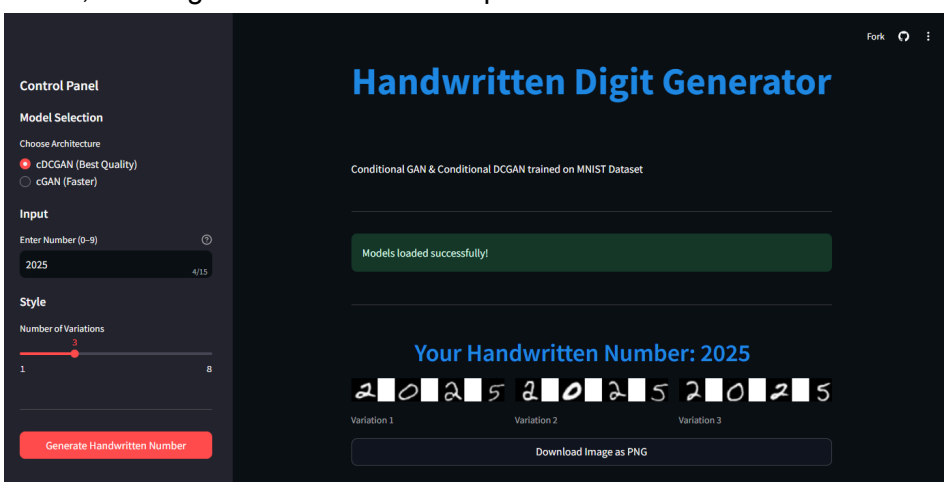
5. Web App Implementation (Streamlit)

To demonstrate the capabilities of the trained cGAN [2] and cDCGAN [3] models, a user-friendly web application was developed using Streamlit. This application allows users to dynamically generate handwritten numbers based on their input, visually comparing the output quality and performance of the two different architectures.

5.1 Architecture and Features

The web application is structured around a few key features that leverage the underlying PyTorch models:

- **Model Loading:** The pre-trained weights for both the fully-connected cGAN [2] and the convolutional cDCGAN [3] were loaded into the application using Streamlit's resource caching mechanism (@st.cache_resource). This ensures that the large model files are loaded only once when the application starts, significantly improving generation speed for subsequent requests.
- **Model Selection:** A radio button in the sidebar allows users to switch between the two models: cDCGAN [3] (Best Quality) and cGAN [2] (Faster). This highlights the trade-off between image realism (cDCGAN [3]) and generation speed (cGAN) observed during training.
- **Number Input and Parsing:** Users can input a number (a sequence of digits). The application parses this string into a sequence of individual digits.
- **Variational Generation:** A slider control allows the user to specify the "Number of Variations" (up to 8). For each variation, the application generates a unique random noise vector (z), ensuring that the resulting handwritten digits are stylistically distinct, even when conditioned on the same label.
- **Image Combination:** For multi-digit inputs (e.g., "2025"), the application iteratively generates each digit independently and then combines them into a single, cohesive image with padding. This demonstrates the model's ability to act as a core component for synthetic image composition.
- **Download Functionality:** The generated number image is provided with a download button, allowing users to save the output as a PNG file.



6. Discussion

6.1 Discussion

- **cDCGAN [3] Superiority:** The cDCGAN [3] produced significantly higher-quality, sharper images and exhibited greater training stability, characterized by smooth loss curves. This confirms that convolutional layers are better suited for learning the spatial hierarchies and fine-grained patterns present in handwritten digits.
    - The conditional mechanism was highly effective, with the generator reliably creating digits corresponding to the correct label.

- **cGAN as a Baseline:** The cGAN [2], while training much faster (approx 16.8 seconds per epoch), generated blurry and inconsistent images. The unstable, fluctuating loss curves confirmed the difficulty of training an adversarial network without convolutional feature extraction.

6.2 Limitations

- **Training Time (cDCGAN [3]):** While the convolutional model yielded better results, it took significantly longer to train (approx 70 seconds per epoch). This limits the ability to rapidly iterate on architecture changes or scale to larger datasets.
- **Artifacts and Consistency:** Minor artifacts and structural inconsistencies were still present in the generated images, especially in the boundaries or curves of certain digits.
- **Small Dataset:** Training was performed exclusively on the MNIST dataset, which features small, centralized, black-and-white digits. The models may not generalize well to more complex, real-world handwritten text.

6.3 Future Work

- **Advanced Conditioning:** Implement Conditional Batch Normalization (CBN) to integrate the class label directly into the BatchNorm layers of the Generator. This method often leads to a cleaner separation of style (from noise) and content (from label), potentially boosting image quality.
- **Larger Datasets:** Retrain the cDCGAN [3] model on the EMNIST dataset, which includes both handwritten digits and letters. This would significantly expand the model's utility to generate entire words and more complex inputs.
- **Perceptual Loss Integration:** Incorporate L1 or L2 loss alongside the standard adversarial loss (BCELoss). Adding a pixel-wise loss can help force the generator to produce sharper, more detailed output, mitigating the blurriness inherent in some GAN [1]outputs.

7. Contributions

Farooq (Project Lead)	Designed and built the fully-connected cGAN [2] architecture (the baseline model). Managed the comparison of training times and stability between the two models.	Established the entire Streamlit application framework, including the basic layout, custom styling, and managing the final hosting/deployment.
Wasim	Developed the Conditional Deep Convolutional GAN [1]architecture.Handled data preparation for the cDCGAN [3].	Implemented the model switching logic, allowing the user to select between cGAN [2] and cDCGAN [3], and ensured

		efficient model loading using Streamlit's caching.
Hateem	Managed the core training process, including setting up the Binary Cross-Entropy (BCE) loss function and correctly applying the adversarial updates for both the Discriminator and Generator. Configured the Adam optimizer and all learning rate parameters.	Designed and coded the user controls in the sidebar, including the number input field, the variation slider, and ensuring all inputs are correctly validated.
Umer	Responsible for training monitoring and data logging, including tracking and saving per-epoch timing and the loss histories for both models. Also implemented the routine to save generated sample images during training for qualitative evaluation.	Developed the essential image composition logic to stitch single digits together into multi-digit numbers (combine_digits function). Implemented the final PNG download feature for the generated images.

References

[1] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative Adversarial Nets. In *Advances in Neural Information Processing Systems (NIPS 2014)* (pp. 2672-2680).

[2] Mirza, M., & Osindero, S. (2014). Conditional Generative Adversarial Nets. *arXiv preprint arXiv:1411.1784*.

[3] Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. *arXiv preprint arXiv:1511.06434*.

[4] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

[5] Paszke, A., Gross, S., Massa, F., Lerer, A., ... Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NIPS 2019)*.

[6] Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. Presented at the *International Conference on Learning Representations (ICLR 2015)*.

[7] Streamlit: The fastest way to build data apps. (n.d.). [Online]. Available: https://streamlit.io/ (Accessed: Nov. 2025).